Documento de implementación RDBMS – CafeLudoteca



Integrantes: Santiago Reina Aguilar, Mayra Alejandra Yanes, Michael Andres Ortiz Giral

Docente: Christian Duarte

Bogota

Universidad el Bosque

Facultad de Ingenieria

Programa: Ingenieria de Sistemas

Introducción y alcance

Este documento describe cómo se implementa en MySQL la base de datos relacional de CafeLudoteca: tablas, llaves, restricciones y el acceso desde la aplicación Java.

Cubre diez tablas que ya tienen repositorio en el proyecto: CLIENTE, CLIENTE_FRECUENTE, SOCIO, MEMBRESIA, BENEFICIO, TELEFONO, EMPLEADO, JUEGO_MESA, MESA y RESERVA. Otras cinco entidades (Categoria, ElementoJuego, Expansion, Prestamo y UnidadJuego) existen en el modelo Java, pero todavía no tienen repositorio ni relaciones definidas; se listan al final del diccionario de datos.

El documento sirve como referencia para crear la base, cargar datos de prueba y revisar que el código y el modelo coincidan.

Motor de base de datos y herramientas

El motor es MySQL, administrado con MySQL Workbench y la aplicación Java se conecta por JDBC.

Componente	Versión y uso en el proyecto
MySQL	*Motor relacional; la base se llama proyecto_db1*
MySQL Workbench	*Diseño del modelo, ejecución de scripts y consulta de datos*
MySQL Connector/J	*9.5.0; driver JDBC que conecta Java con MySQL*
HikariCP	*7.1.0; pool de conexiones*
Java y Maven	*Java 25; el proyecto se empaqueta como WAR*
Jakarta REST (JAX-RS)	*3.1.0; controladores REST, con Jackson 2.20.1 para el JSON*
Lombok	*1.18.42; genera getters, setters y constructores de las entidades*
JUnit	*4.13.2; pruebas unitarias*

La conexión se centraliza en la clase ConnectionBD, que usan todos los repositorios y que abre una conexión nueva con DriverManager en cada llamada. HikariCP ya está en el pom.xml; al activarlo solo cambia esa clase.

Modelo relacional

CLIENTE es la tabla central: de ella dependen cinco tablas. MEMBRESIA tiene beneficios y MESA tiene reservas; EMPLEADO y JUEGO_MESA no tienen llaves foráneas.

Tabla	Llave primaria	Llave foránea	Referencia
CLIENTE	*documento*	—	—
CLIENTE_FRECUENTE	*documento*	*documento*	*CLIENTE.documento*
SOCIO	*documento*	*documento*	*CLIENTE.documento*
MEMBRESIA	*id_membresia*	*documento*	*CLIENTE.documento*
BENEFICIO	*id_membresia* + *id_beneficio*	*id_membresia*	*MEMBRESIA.id_membresia*

Tabla	Llave primaria	Llave foránea	Referencia
TELEFONO	*documento + telefono*	*documento*	*CLIENTE.documento*
EMPLEADO	*documento*	—	—
JUEGO_MESA	*num_serie*	—	—
MESA	*id_mesa*	—	—
RESERVA	*id_reserva*	*documento, id_mesa*	*CLIENTE.documento, MESA.id_mesa*

Las relaciones son de uno a uno entre CLIENTE y CLIENTE_FRECUENTE o SOCIO, y de uno a muchos en los demás casos: un cliente tiene varias membresías, teléfonos y reservas, una membresía tiene varios beneficios y una mesa tiene varias reservas.

Entidades sin repositorio

Estas entidades existen en el modelo Java, pero sus tablas y relaciones se documentan cuando se defina el esquema.

Entidad	Atributos
Categoria	*numCategoria, categoria*
ElementoJuego	*idElementoJuego, elementoJuego*
Expansion	*idExpansion, nombre, descripcion, contenidoAdicional*
Prestamo	*idPrestamo, fechaSalida, estadoComponentes, tipoPrestamo, fechaDevolucion*
UnidadJuego	*codigo, estado, observaciones*

Llaves, restricciones e integridad referencial

La base rechaza cualquier fila que apunte a un registro padre inexistente, y el orden de inserción y borrado debe respetar eso.

- Cada tabla tiene llave primaria; BENEFICIO y TELEFONO la tienen compuesta.
- Las llaves foráneas apuntan a CLIENTE, MEMBRESIA o MESA, según la tabla de relaciones.
- Las columnas marcadas como "No" en el diccionario llevan NOT NULL.
- La tarifa y los puntos no pueden ser negativos (CHECK).
- Las llaves foráneas deben tener el mismo tipo que la columna que referencian: BENEFICIO.id_membresia es VARCHAR(10), igual que MEMBRESIA.id_membresia.

Orden de inserción:

1. CLIENTE, MESA, EMPLEADO y JUEGO_MESA, que no dependen de otras tablas
2. CLIENTE_FRECUENTE, SOCIO, MEMBRESIA, TELEFONO y RESERVA

3. BENEFICIO, porque depende de MEMBRESIA

El borrado va en orden inverso. Si se borra un CLIENTE con datos dependientes, MySQL lo rechaza, salvo que las llaves foráneas se definan con ON DELETE CASCADE. Aquí se dejan sin cascada para evitar borrados accidentales.

En la aplicación, un INSERT que viole una llave devuelve false en el repositorio y null en el servicio, que el controlador convierte en una respuesta 400.

Acceso a datos desde la aplicación

Cada petición HTTP atraviesa cinco capas antes de llegar a MySQL, y cada capa tiene una sola responsabilidad.

1. **Controlador** (XController): recibe la petición JAX-RS y devuelve la respuesta HTTP.
2. **Servicio** (XService): valida los datos y coordina el repositorio.
3. **Mapper** (XMapper): convierte entre DTO y entidad.
4. **Repositorio** (XRepositoryJDBC): ejecuta el SQL con PreparedStatement.
5. **Conexión** (ConnectionBD): entrega la conexión a MySQL.

Todas las consultas usan parámetros ?, lo que evita la inyección de SQL, y las conexiones se cierran con try-with-resources.

Entidad	*Ruta REST*	*Consultas disponibles*
Cliente	*/cliente*	*por nombre, por documento*
ClienteFrecuente	*/clienteFrecuente*	*por nivel de fidelidad, por documento*
Socio	*/socio*	*por fecha de afiliación, por documento*
Membresia	*/membresia*	*por documento, por estado, por identificador*
Beneficio	*/beneficio*	*por membresía, por membresía y beneficio*
Telefono	*/telefono*	*por documento, por documento y número*
Empleado	*/empleados*	*por nombre, por documento*
JuegoMesa	*/juegoMesa*	*por nombre, por número de serie*
Mesa	*/mesas*	*por temática, por identificador*
Reserva	*/reservas*	*por documento, por identificador*

Todas permiten crear, consultar, actualizar y borrar (POST, GET, PUT, DELETE).